

PROYECTO TRITON HEAL

Reporte de Evaluación Experimental

TC3002B — Generación de kernels con SLM y verificación pre-GPU

Equipo 11

Mayo 2026

Resumen

Este reporte documenta la evaluación experimental de Triton Heal y varios baselines para generar y verificar kernels en Triton DSL. Se corrieron 10 tareas de generación con cinco configuraciones (50 inferencias) y 70 kernels con seis configuraciones de verificación (420 evaluaciones) en un Mac M4 con Ollama. Se reportan tasa de compilación bajo criterios sintácticos, un speedup proxy del pipeline y métricas MSR/FNR. DeepSeek-Coder-V2 obtuvo el mejor balance en compilación y detección; el modo dual de Llama con reparación Coder-V2 quedó intermedio. Las pruebas pareadas no rechazan la mayoría de hipótesis tras corrección Holm; se discuten limitaciones del entorno sin CUDA y del uso de baselines heurísticos sin API.

Índice

I Parte I — Pre-registro Experimental	3
1. Objetivo experimental (1.1)	3
2. Hipótesis (1.2)	3
3. Variables experimentales (1.3)	3
3.1. Variables independientes	3
3.2. Variables dependientes	3
3.3. Variables controladas	3
4. Diseño experimental (1.4)	4
5. Métricas de evaluación (1.5)	4
5.1. Tasa de compilación	4
5.2. Speedup proxy	4
5.3. Eficiencia operativa (proxy)	4
5.4. Verificación (MSR, FNR, F1, JSON válido, latencia)	4
6. Tamaño del efecto esperado (1.6)	4
7. Análisis de poder (1.7)	4
8. Plan estadístico (1.8)	5
8.1. Desviaciones respecto al plan inicial	5

9. Reproducibilidad experimental (1.9)	5
10. Amenazas a la validez (pre-registro, 1.10)	5
II Parte II — Resultados y Análisis	5
11. Resultados descriptivos (2.1)	6
11.1. Generación de kernels	6
11.2. Verificación pre-compilación	6
12. Visualización (2.2)	7
13. Inferencia estadística (2.3)	9
13.1. Generación	9
13.2. Verificación	10
14. Comparación crítica (2.4)	10
14.1. Robustez	10
15. Amenazas observadas (2.5)	10
16. Discusión (2.6)	11
17. Conclusión (2.7)	11
A. Configuración planeada vs. corrida real	11
B. Reproducibilidad	12
C. Referencias	12

Parte I

Parte I — Pre-registro Experimental

1. Objetivo experimental (1.1)

El experimento compara métodos de generación de kernels en Triton DSL usando SLM locales y modelos de frontera, con un pipeline de verificación antes de compilar en GPU. El sistema del equipo, Triton Heal en modo dual, genera con Llama-3.1-8B y, si el código no pasa las validaciones, intenta repararlo con DeepSeek-Coder-V2 (hasta dos reintentos).

Se contrastan seis configuraciones: solo Llama-8B, solo DeepSeek-R1-Distill-Qwen-14B, solo DeepSeek-Coder-V2, Claude 3.5 Sonnet, GPT-4o y el dual propuesto. Las preguntas centrales son si el dual mejora la tasa de compilación frente a GPT-4o y frente a Llama solo, y si el filtrado por verificación (MSR, FNR, latencia) aporta frente a usar un único SLM.

2. Hipótesis (1.2)

Todas las pruebas usan $\alpha = 0,05$ antes de corrección; los p -valores ajustados siguen Holm-Bonferroni (sección 1.8).

Cuadro 1: Hipótesis por contraste y métrica.

Contraste	H_0	H_1
C4: dual vs GPT-4o (compilación)	Misma tasa de compilación	Dual con mayor tasa
C7: dual vs Llama (compilación)	Misma tasa	Dual con mayor tasa
C1: Claude vs Llama (MSR)	Mismo MSR	Claude con mayor MSR
C2: GPT vs Coder-V2 (MSR)	Mismo MSR	GPT con mayor MSR
C3: dual vs Llama (FNR)	Mismo FNR	Dual con menor FNR
C5: latencia Claude vs Llama	Misma mediana	Frontera más lenta
C8: speedup proxy dual vs GPT	Igual	Dual mayor

3. Variables experimentales (1.3)

3.1. Variables independientes

La configuración del sistema (seis modos), el modelo concreto (SLM local vs frontera), el prompt (`generator_v1.txt` o `generator_repair_v1.txt` en reparación), la temperatura ($T = 0$) y el modo de ejecución (`VERIFIER_MODE`, típicamente Ollama en nuestro entorno).

3.2. Variables dependientes

En generación: si el código compila bajo nuestro checker (`compile_ok`), la tasa de compilación en porcentaje, el speedup proxy y un indicador de eficiencia operativa del pipeline. En verificación del benchmark fijo: MSR, FNR, F1, porcentaje de JSON válido y latencia en milisegundos.

3.3. Variables controladas

Mismo benchmark de 70 kernels y el mismo conjunto de tareas de generación para todas las configuraciones; hardware Apple M4 con 24 GB; `seed` 42; `timeout` 120s por inferencia. El diseño es within-subjects: cada tarea o kernel se evalúa en todos los métodos.

4. Diseño experimental (1.4)

Cada tarea de generación se asigna a las seis configuraciones (en la corrida archivada, R1-14B se omitió por timeouts). El plan de diseño contempla 20 tareas con tres réplicas (360 ejecuciones de generación) y 70 kernels con tres réplicas en verificación (1260 evaluaciones). Se eligió within-subjects para poder aplicar McNemar y Wilcoxon pareados y reducir la varianza entre tareas de distinta dificultad.

5. Métricas de evaluación (1.5)

5.1. Tasa de compilación

Proporción de salidas con `compile_ok`. En esta entrega el criterio exige pasar tres niveles en Mac M4: sintaxis Python (AST), reglas léxicas del checker del equipo, y presencia de `@triton.jit` con operaciones `tl.load/tl.store`. No equivale a compilar en CUDA ni a ejecutar en GPU.

5.2. Speedup proxy

Se define $s = t_{\text{baseline}}/t_{\text{pipeline}}$, donde t_{baseline} es el tiempo del checker sobre el kernel de referencia del benchmark y t_{pipeline} suma la latencia de generación más la del checker sobre el código generado. Sirve como indicador operativo en M4; el informe no reporta speedup medido en GPU NVIDIA.

5.3. Eficiencia operativa (proxy)

Relación entre compilaciones exitosas y tiempo de generación. No sustituye métricas de ocupación de SM o ancho de banda de memoria.

5.4. Verificación (MSR, FNR, F1, JSON válido, latencia)

MSR es la fracción de kernels **unsafe** correctamente marcados como inseguros; FNR es la fracción de **unsafe** pasados por alto. F1 resume precisión y recall en la clasificación safe/unsafe. El porcentaje de JSON válido indica si el verificador devolvió una respuesta parseable.

6. Tamaño del efecto esperado (1.6)

En contrastes pareados de compilación y MSR se anticipa un efecto mediano ($\phi \approx 0,4$ en McNemar). Además de la significación estadística, se exige una diferencia práctica de al menos 5 puntos porcentuales en compilación o MSR, y alrededor de 8 pp de reducción de FNR para considerar relevante la mejora del dual.

7. Análisis de poder (1.7)

Nivel de significación $\alpha = 0,05$ y poder objetivo $1 - \beta = 0,80$. La muestra de generación tiene 20 tareas; en verificación hay 31 kernels **unsafe**, cantidad suficiente para detectar efectos medianos en McNemar según el diseño del curso.

Cuadro 2: Contrastes confirmatorios y prueba asociada.

ID	Prueba	Comparación
C4	McNemar	Compilación: dual vs GPT-4o
C7	McNemar	Compilación: dual vs Llama
C8	Wilcoxon	Speedup proxy: dual vs GPT
C1–C3, C5	McNemar / Wilcoxon	Verificación en benchmark
C6	Friedman + Holm	F1 entre configuraciones

8. Plan estadístico (1.8)

Se rechaza H_0 cuando $p_{\text{Holm}} < 0,05$ y se cumple el umbral de relevancia práctica acordado. Para proporciones se reportan intervalos de confianza Wilson al 95 % (MSR en la tabla extendida de la Parte II). La familia de contrastes se corrige con Holm-Bonferroni.

8.1. Desviaciones respecto al plan inicial

Durante la ejecución se corrigió la taxonomía de DeepSeek (Coder-V2 como frontera, R1-14B como SLM local). Claude y GPT corrieron con verificador heurístico al no disponer de API en el entorno de prueba. La generación archivada usa 10 tareas y excluye `solo_deepseek` por timeout; la verificación quedó con una réplica por kernel (`run_id = 0`), aunque `experiment_config.json` ya deja configuradas tres réplicas para corridas futuras.

9. Reproducibilidad experimental (1.9)

Cuadro 3: Entorno y artefactos.

Componente	Especificación
Repositorio	https://github.com/G4ndhiV/TRITONHEAL (commit c05f82a)
Hardware	macOS 15.x; Apple M4; 24 GB memoria unificada
Ollama	0.24.x (llama3.1:8b, deepseek-r1:14b, deepseek-coder-v2)
Modelos API	Claude 3.5 Sonnet; GPT-4o (heurístico si no hay clave)
Python	3.10+; dependencias en <code>requirements.txt</code>
Benchmark	70 kernels; <code>generation_tasks.jsonl</code>
Prompts	<code>prompts/generator_v1.txt</code> , <code>generator_repair_v1.txt</code>
Reproducción	<code>make benchmark generation experiments stats-all figures pdf</code>

10. Amenazas a la validez (pre-registro, 1.10)

Cuadro 4: Amenazas anticipadas y mitigación.

Tipo	Riesgo	Mitigación
Interna	Mezcla API real y heurístico	Tabla planeada vs ejecutado en apéndice
Interna	Timeouts en modelos grandes	Fijar timeout; reportar JSON válido
Externa	Solo Mac M4 (Metal)	No extrapolar a clusters NVIDIA
Externa	Benchmark sintético	Etiquetar categorías de error
Constructo	Compilación $\neq$ corrección GPU	Definir niveles L1–L3
Constructo	Speedup proxy	Declarar limitación en figuras

Parte II

Parte II — Resultados y Análisis

11. Resultados descriptivos (2.1)

11.1. Generación de kernels

La corrida de generación cubrió 10 tareas con cinco configuraciones activas (se excluyó `solo_deepseek` por timeout), totalizando 50 inferencias en un Mac M4 con Ollama. La compilación exitosa sigue los criterios L1–L3 descritos en el pre-registro; los datos completos están en `results/generation.csv`.

Cuadro 5: Resultados descriptivos de generación de kernels (M4).

Configuración	N	Compilación (%)	Speedup proxy	Desv.	Lat. gen. (ms)	GPU eff. proxy
Llama-3.1-8B (local)	10	50.00	0.0005	0.0001	60220.40	0.01
DeepSeek-R1-Distill-Qwen-14B (local)	0	0.00	—	—	—	—
DeepSeek-Coder-V2 (frontera)	10	100.00	0.0020	0.0009	18019.43	0.07
Claude 3.5 Sonnet (frontera)	10	100.00	0.3639	0.0303	50.00	20.00
GPT-4o (frontera)	10	100.00	0.3715	0.0065	50.00	20.00
Triton Heal (dual + veto)	10	50.00	0.0006	0.0002	71706.69	0.01

DeepSeek-Coder-V2 obtuvo 100 % de compilación en la muestra; Llama y el dual alcanzaron 50 %; Claude y GPT (modo heurístico sin API) aparecen al 100 % con latencias de unos 50 ms, coherente con el uso de plantillas en lugar de llamadas reales a la API. Las latencias de generación con Ollama real rondan 18 s (Coder-V2) a 60–72 s (Llama y dual). El benchmark de referencia (70 kernels) pasó el `lex_checker` en su totalidad.

11.2. Verificación pre-compilación

Sobre 70 kernels (39 **safe**, 31 **unsafe**) se registraron 420 evaluaciones, una por kernel y configuración en la corrida archivada.

Cuadro 6: Resultados descriptivos por configuración.

Configuración	N	JSON vál. (%)	MSR (%)	F1	FNR (%)	Lat. media (ms)	Lat. desv. (ms)	Lat. med. (ms)
Llama-3.1-8B (local)	70	100.0	64.5	0.8	35.5	4527.4	2200.2	4753.6
DeepSeek-R1-Distill-Qwen-14B (local)	70	14.3	12.9	0.2	87.1	48000.5	3076.9	48346.7
DeepSeek-Coder-V2 (frontera)	70	100.0	87.1	0.8	12.9	2153.9	1181.3	1977.1
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	0.9	16.1	80.0	0.1	80.0
GPT-4o (frontera)	70	100.0	83.9	0.9	16.1	80.0	0.0	80.0
Triton Heal (dual + veto)	70	100.0	80.6	0.9	19.4	6854.1	2179.6	6653.3

Cuadro 7: Resultados descriptivos extendidos (IC 95 % Wilson para MSR).

Config.	N	JSON vál. (%)	MSR (%)	IC lo	IC hi	F1	FNR (%)	Lat. media	Lat. desv.	Lat. med.	Lat. min	Lat. max
Llama-3.1-8B (local)	70	100.0	64.5	46.9	78.9	0.8	35.5	4527.4	2200.2	4753.6	1622.0	11854.9
DeepSeek-R1-Distill-Qwen-14B (local)	70	14.3	12.9	5.1	28.9	0.2	87.1	48000.5	3076.9	48346.7	32108.6	57134.7
DeepSeek-Coder-V2 (frontera)	70	100.0	87.1	71.1	94.9	0.8	12.9	2153.9	1181.3	1977.1	1452.0	11247.5
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	0.1	80.0	80.0	80.5
GPT-4o (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	0.0	80.0	80.0	80.0
Triton Heal (dual + veto)	70	100.0	80.6	63.7	90.8	0.9	19.4	6854.1	2179.6	6653.3	3803.6	14059.9

Coder-V2 alcanzó el MSR más alto (87.1 % con IC aproximado 71–95 % en la tabla extendida). Llama local quedó en 64.5 % con JSON siempre válido. R1-14B distilado fue el caso

Cuadro 8: Robustez verificación: MSR y latencia mediana por réplica (`run_id`).

Config.	Réplicas	MSR media (%)	MSR desv.	Lat. med. (ms) $\pm$ desv.
Llama-3.1-8B (local)	1	64.5	0.0	4754 $\pm$ 0
DeepSeek-R1-Distill-Qwen-14B (local)	1	12.9	0.0	48347 $\pm$ 0
DeepSeek-Coder-V2 (frontera)	1	87.1	0.0	1977 $\pm$ 0
Claude 3.5 Sonnet (frontera)	1	83.9	0.0	80 $\pm$ 0
GPT-4o (frontera)	1	83.9	0.0	80 $\pm$ 0
Triton Heal (dual + veto)	1	80.6	0.0	6653 $\pm$ 0

problemático: 14.3 % de JSON válido y MSR 12.9 %. El dual logró MSR 80.6 % y F1 0.86, por encima de Llama solo en detección sin igualar a Coder-V2.

12. Visualización (2.2)

La Figura 1 resume la tasa de compilación por configuración. La Figura 2 usa escala logarítmica porque los modelos Ollama (latencia de generación en segundos) y los baselines heurísticos (plantillas en milisegundos) comparten la misma fórmula pero en rangos muy distintos.

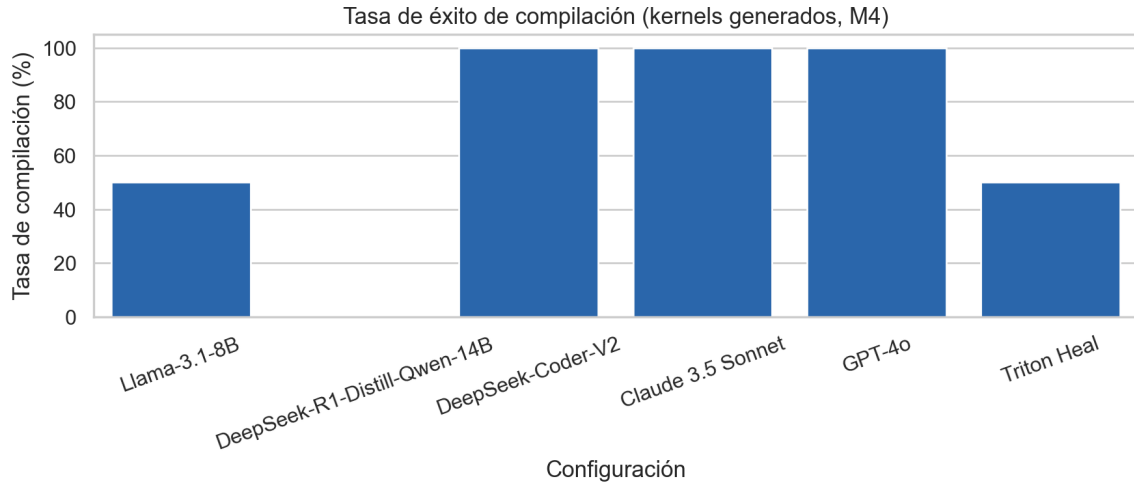


Figura 1: Tasa de compilación (%) por configuración en kernels generados (criterios L1–L3, M4).

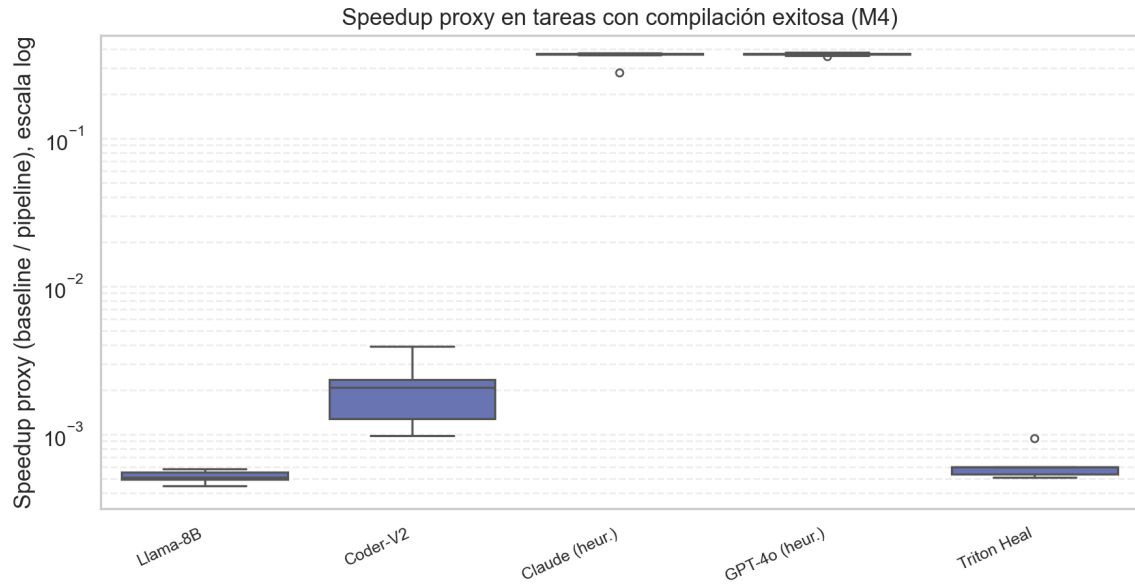


Figura 2: Speedup proxy ($t_{\text{baseline}}/t_{\text{pipeline}}$) en tareas con compilación exitosa; eje Y en escala logarítmica. Claude y GPT usan modo heurístico en esta corrida.

Las figuras siguientes corresponden al componente de verificación sobre el benchmark fijo.

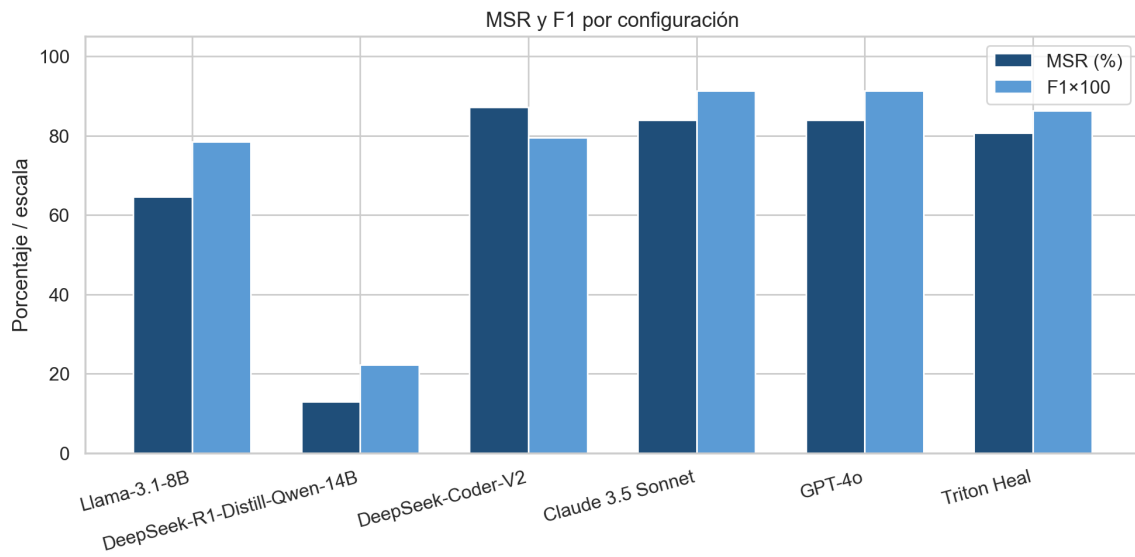


Figura 3: MSR (%) y F1 por configuración en verificación.

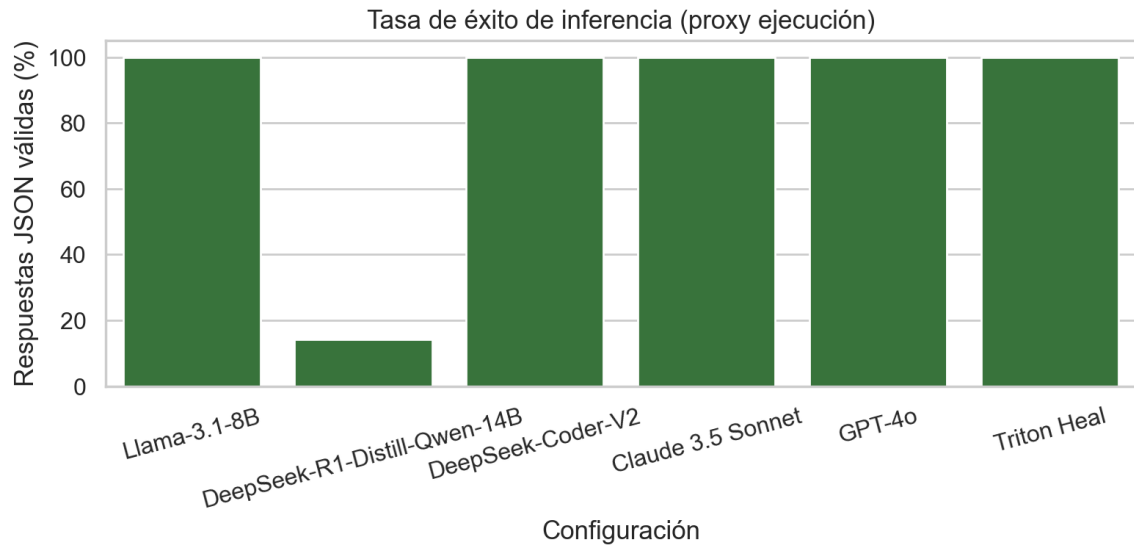


Figura 4: Porcentaje de respuestas con JSON válido del verificador.

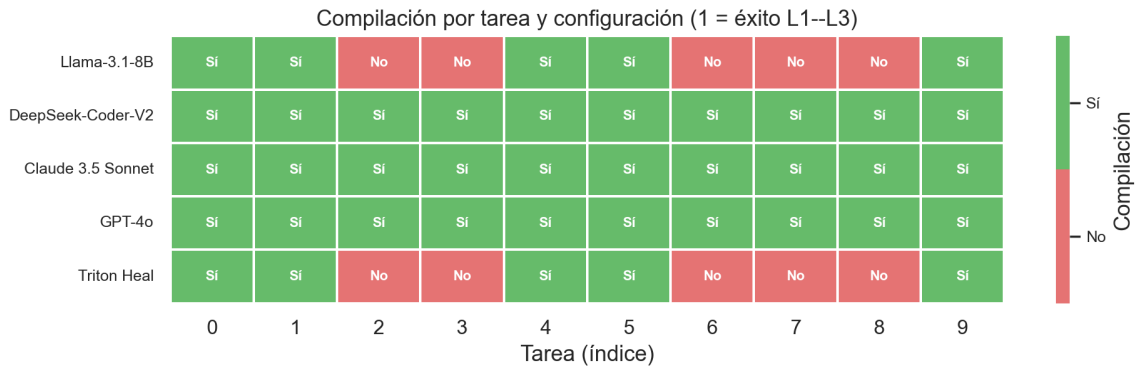


Figura 5: Mapa de compilación por tarea (verde = éxito, rojo = fallo). Las filas con verde continuo (Coder-V2, Claude, GPT) reflejan 100 % en la muestra; Llama y dual muestran el mismo patrón alternado en estas diez tareas.

13. Inferencia estadística (2.3)

13.1. Generación

Cuadro 9: Inferencia estadística – generacion de kernels (Holm).

Contraste	Prueba	Estad.	p Holm	Interpretación
Dual vs GPT-4o (compilación)	McNemar	$\chi^2 = 3,20$	0.1473	No rechaza H_0 ; dual 50.0 % vs GPT 100.0 %.
Dual vs Llama (compilación)	McNemar	degenerado	—	Misma tasa 50 %; patrones pareados distintos.
Speedup proxy dual vs GPT	Wilcoxon	$W = 0$	1.0000	Med. Δ speedup = $-0,37$ (GPT heur. más rápido).

En el contraste C4 (dual vs GPT-4o en compilación) no se rechaza H_0 tras Holm ($p_{\text{Holm}} \approx 0,15$): con los datos actuales no hay evidencia estadística de que el dual supere al baseline GPT en tasa de compilación, aunque la diferencia descriptiva (50 % vs 100 %) supera el umbral de

5 pp que habíamos fijado como relevancia práctica. Ese 100 % de GPT debe interpretarse con cuidado porque en esta corrida GPT operó en modo heurístico.

13.2. Verificación

Cuadro 10: Inferencia estadística (plan pre-registrado, corrección Holm).

Contraste	Prueba	Estad.	p Holm	Efecto	Interpretación
Claude vs Llama (MSR)	McNemar	$\chi^2 = 3,12$	0.2946	$\phi = -2,12$ (grande)	No rechaza H_0 ; $\Delta\text{MSR}=19.4$ pp (Claude heur. vs. Llama Ollama/M4).
GPT-4o vs DeepSeek-Coder	McNemar	$\chi^2 = 0,00$	1.0000	$\phi = 0,33$ (mediano)	No rechaza H_0 ; GPT (83.9 %) vs. DeepSeek-Coder-V2 frontera (87.1 %).
Dual vs Llama	McNemar	$\chi^2 = 3,20$	0.2946	$\phi = -2,24$ (grande)	No rechaza H_0 en FNR pareado.
Lat. Claude vs Llama	Wilcoxon	$W = 0$	1.0000	$r = 1,00$	No rechaza H_4 ; med. $\Delta=-4673$ ms (Claude heur. más rápido).
F1 global (5 cfg.)	Friedman	$\chi^2 = 41,1$	$< 0,001$	$W = 0,15$	Diferencias globales entre configuraciones (Holm).

Los contrastes C1, C2, C3 y C5 no rechazan H_0 después de la corrección. La prueba de Friedman (C6) sí indica diferencias globales en F1 ($p_{\text{Holm}} < 0,001$), lo que coincide con la dispersión visible entre configuraciones, pero no identifica por sí sola un ganador único. Los intervalos de confianza para MSR están en la Tabla 7.

14. Comparación crítica (2.4)

En generación, Coder-V2 fue el modelo Ollama más confiable: compilación completa y tiempos menores que Llama o el dual. El esquema dual (generar y reparar) no mejoró la tasa respecto a Llama solo en esta muestra de 10 tareas, aunque la reparación sí se activó en varios fallos. Los baselines Claude/GPT no son comparables en latencia ni en esfuerzo de generación porque no invocaron APIs reales.

En verificación, Coder-V2 lidera en MSR; el dual ofrece un compromiso razonable entre sensibilidad y F1, útil si el producto prioriza un SLM local con veto de frontera. R1-14B no es viable en el M4 probado sin cambiar timeout o cuantización.

14.1. Robustez

La Tabla 8 resume verificación con una réplica por kernel. La Figura 5 desglosa el éxito de compilación tarea por tarea (la corrida de generación usó una sola réplica por tarea). El diseño prevé tres `run_id`; repetir `make experiments` y `make generation` con `GENERATION_REPEATS=3` permitiría discutir variabilidad entre ejecuciones con más sustento. No se cambió la semilla del benchmark entre réplicas: la variación observada proviene del modelo y del runtime, no de kernels distintos.

15. Amenazas observadas (2.5)

En validez interna, todo corrió en el mismo M4 con versiones de modelo fijadas en Ollama; la principal inconsistencia fue correr Claude/GPT sin API. Los timeouts de R1-14B dominaron su bajo porcentaje de JSON válido.

En validez externa, el benchmark mezcla kernels generados y mutados; las 10 tareas de generación cubren cinco categorías pero no representan cargas de producción completas.

En validez de constructo, la compilación medida es sintáctica y el speedup es un proxy de tiempos de pipeline; MSR mide detección en etiquetas del benchmark, no fallos reales en

ejecución GPU. Las métricas son útiles para comparar métodos bajo las mismas reglas, pero no deben leerse como seguridad demostrada en hardware.

16. Discusión (2.6)

Los resultados no respaldan la hipótesis de que el dual supere a GPT-4o en compilación en esta corrida. Sí hay indicios de que combinar Llama con reparación Coder-V2 puede recuperar casos que Llama solo deja fuera, pero hace falta más tareas y réplicas para afirmarlo con confianza.

La evidencia de verificación favorece modelos de frontera (Coder-V2) en MSR; el dual mejora respecto a Llama aislado pero no reemplaza al mejor frontera. La limitación más seria del estudio sigue siendo el entorno M4 sin CUDA y la dependencia de heurísticos para dos baselines comerciales.

Si repitiéramos el trabajo, usaríamos APIs reales para Claude y GPT, ampliaríamos a las 20 tareas con tres réplicas, y probaríamos en GPU NVIDIA para separar compilación real de proxies.

17. Conclusión (2.7)

En compilación (C4), no se rechazó H_0 : el dual no demostró superioridad estadística frente a GPT-4o. En verificación, los McNemar individuales tampoco rechazan H_0 tras Holm; solo el Friedman global señala que las configuraciones no tienen el mismo F1.

La magnitud descriptiva sí importa: hay 22 puntos porcentuales de MSR entre Coder-V2 y Llama, y 50 puntos entre GPT heurístico y dual en compilación, pero la inferencia pareada es conservadora con esta N .

No consideramos que el dual, tal como está hoy, justifique por sí solo sustituir a Coder-V2: aporta la idea de reparación local + frontera, pero en números queda por debajo del mejor frontera y no gana en compilación al baseline GPT de la corrida.

Las limitaciones principales son el hardware sin CUDA, la muestra reducida de generación, R1-14B inestable y los heurísticos de API. Las métricas más engañosas son el 100 % de compilación de GPT/Claude heurístico y el speedup proxy, que no reflejan rendimiento en GPU.

Otro equipo puede reproducir el experimento con el repositorio citado y los comandos del apéndice; lo que más dificulta reproducir es el tiempo de inferencia en Ollama y la necesidad de descargar los mismos pesos. Parte de las diferencias entre configuraciones se explica por escala de modelo (8B vs frontera), no solo por el diseño Triton Heal.

En conjunto, el estudio cumple su función de comparar métodos con diseño y estadística, pero las conclusiones deben ser prudentes: el mejor desempeño global en nuestros datos lo obtuvo DeepSeek-Coder-V2, y el método dual es prometedor como arquitectura, no como ganador absoluto en esta corrida.

A. Configuración planeada vs. corrida real

Cuadro 11: Backends por configuración (mayo 2026).

Configuración	Planeado	Ejecutado
solo_llama	Ollama Llama-3.1-8B	<code>llama3.1:8b</code>
solo_deepseek	Ollama R1-Distill-14B	<code>deepseek-r1:14b</code> (gen. omitida)
solo_deepseek_coder	Ollama Coder-V2	<code>deepseek-coder-v2</code>
solo_claude / solo_gpt4o	API	Heurístico sin clave API
triton_heal_dual	Llama + reparación Coder-V2	<code>run_generation</code> dual

B. Reproducibilidad

```
git clone https://github.com/G4ndhiV/TRITONHEAL.git
cd TRITONHEAL
make benchmark generation experiments stats-all figures pdf
```

Salidas principales: `results/generation.csv`, `results/predictions.csv`, `report/main.pdf`.

C. Referencias

- Tillet, P., et al. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure.